used to display a field by adding a `template_name` argument to any field object in your `Form` class:

```
class CommentForm(forms.Form):
    name = forms.CharField(template_name="custom_input_field.html")
    ...
```

Specify the use of a custom template to render your field.

This means you can build Bootstrap-style widgets as field templates and reuse them in your forms. If you're going to customize a field, it is best to start with Django's own template as a base. You can find it in the Django package under forms/templates/django/forms/field.html.

7.2 Django ModelForm

It is quite common for there to be a link between the data you ask your users to submit and the information you store in your database. To reduce the amount of code you write, Django includes the `ModelForm` class. This class extends from `Form` and automatically creates fields based on the structure of an associated model class.

It is finally time to implement Nicky's original idea: ads for musicians seeking bands and bands seeking musicians. Ultimately, the goal is to use a `ModelForm` to get your user's input, but there is a bit of a journey to get there. First off, you'll need somewhere to store the ad, which means a new model class. Along the way to implementing the `ModelForm`, you'll learn some more about ORM fields, how to validate multifield interdependencies, and how to use the undocumented `Truncate` class to show part of some text.

7.2.1 Validating fields in a model

Before providing a self-service feature for your users, like writing a classified ad, it is good practice to get any new model working in the Admin first. This provides a way for you to test out your thoughts before exposing it to your users.

The `SeekingAd` model stores a classified ad written by a user, and its key component is a `TextField` containing the ad's content. Ads can be either written by musicians seeking a band or bands seeking musicians. To capture that, you need a foreign relationship to a `Musician` object and a second one to a `Band` object. You also need to store which kind of ad it is; you can do this in several ways. A `BooleanField` could be used, where `True` means *musician seeking band*, and `False` means *band seeking musician*, but then, you have to remember what `True` means. A better choice is to use a `CharField` and store `"M"` when seeking a musician, or `"B"` when seeking a band.

The `CharField` takes an optional argument called `choices` that restricts the values allowed in the field. By using `choices`, you can guarantee that only an `"M"` or `"B"` is stored. The `choices` argument takes a tuple-of-tuples structure, where each `tuple` defines a choice, and the `tuple` within the choice pairs a value with a human-readable label, as shown in the following example: